

Felipe Gallego Rengifo

Jorge Andres Lopez Lopez

Ingenieria en sistemas

Programacion móvil

Raul Gaviria

Universidad libre seccional Pereira

05/06/26

1. Introducción

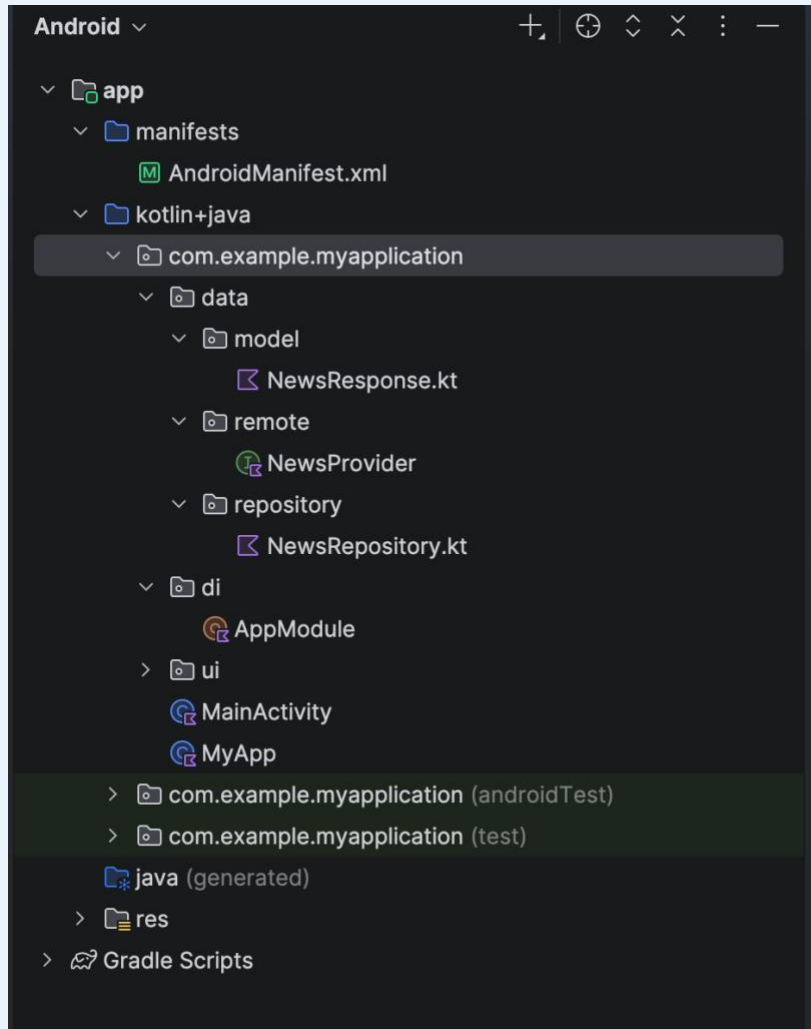
Este documento presenta el desarrollo de una aplicación Android utilizando las tecnologías modernas recomendadas por Google. La app consume la API REST de NewsAPI para mostrar noticias en tiempo real, aplicando el patrón de arquitectura MVVM (Model-View-ViewModel) con Jetpack Compose para la interfaz de usuario, Hilt para la inyección de dependencias, Retrofit para las peticiones de red, y pruebas unitarias con MockWebServer.

2. Arquitectura de la Aplicación

La aplicación sigue la arquitectura moderna recomendada por Android, compuesta por tres capas principales:

2.1 Capa de Datos (Data Layer)

Contiene toda la lógica de negocio y las fuentes de datos. Incluye los modelos de datos, la interfaz de Retrofit (NewsProvider), y los repositorios que abstraen el origen de los datos del resto de la app.



2.2 Capa de Dominio (Domain Layer)

Capa opcional que encapsula la lógica de negocio compleja. En este proyecto se utilizan casos de uso para aislar operaciones específicas y facilitar la reutilización desde múltiples ViewModels.

2.3 Capa de UI (UI Layer)

Responsable de mostrar los datos al usuario. Está compuesta por las pantallas Composable y los ViewModels que exponen el estado de la UI de forma reactiva mediante StateFlow.

3. Configuración de Dependencias

Se configuraron los siguientes plugins y dependencias en los archivos build.gradle del proyecto:

build.gradle (nivel raíz) — Plugins:

```
// Top-level build file where you can add configuration options common to all sub-projects/modules

plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.hilt) apply false
    alias(libs.plugins.ksp) apply false
}
```

build.gradle (módulo app) — Dependencias:

```

// Imágenes
implementation(libs.coil.compose)

// Navegación
implementation(libs.navigation.compose)

// Retrofit (consumo de API REST)
implementation(libs.retrofit)
implementation(libs.retrofit.converter.gson)
implementation(libs.okhttp.logging)

// Hilt (inyección de dependencias)
implementation(libs.hilt.navigation.compose)
implementation(libs.hilt.android)
ksp(libs.hilt.compiler)

// Tests
testImplementation(libs.junit)
androidTestImplementation(libs.androidx.junit)
androidTestImplementation(libs.hilt.android.testin
kspAndroidTest(libs.hilt.compiler)
testImplementation(libs.okhttp.mockwebserver)
testImplementation(libs.kotlinx.coroutines.test)

```

4. Modelos de Datos

Se crearon las data classes necesarias para deserializar la respuesta JSON de la API de NewsAPI. La respuesta principal es NewsResponse, que contiene una lista de objetos Article, cada uno con su fuente Source.

```
package com.example.myapplication.data.model
```

2 Usages

```
data class NewsResponse(  
    val status: String,  
    val totalResults: Int,  
    val articles: List<Article>  
)
```

9 Usages

```
data class Article(  
    val source: Source,  
    val author: String?,  
    val title: String,  
    val description: String?,  
    val url: String,  
    val urlToImage: String?,  
    val publishedAt: String,  
    val content: String?  
)
```

1 Usage

```
data class Source(  
    val id: String?,  
    val name: String  
)
```

5. NewsProvider y Repository

La interfaz NewsProvider define los endpoints de Retrofit. El Repository abstrae la fuente de datos y expone los datos al resto de la aplicación a través de su interfaz.

```

package com.example.myapplication.data.remote

import ...

// Usages
interface NewsProvider {
    1 Usage
    @GET(value = "v2/top-headlines")
    suspend fun getTopHeadlines(
        @Query(value = "country") country: String = "us",
        @Query(value = "apiKey") apiKey: String = "281ec604c8d74c8ab3e62cea4e159f91"
    ): NewsResponse
}

```

```

package com.example.myapplication.data.repository

import com.example.myapplication.data.model.Article
import com.example.myapplication.data.remote.NewsProvider

// Interfaz
6 Usages 1 Implementation
interface NewsRepository {
    2 Usages 1 Implementation
    suspend fun getTopHeadlines(): List<Article>
}

// Implementación
3 Usages
class NewsRepositoryImpl(
    private val newsProvider: NewsProvider
) : NewsRepository {
    override suspend fun getTopHeadlines(): List<Article> {
        return newsProvider.getTopHeadlines().articles
    }
}

```

6. Inyección de Dependencias con Hilt

Hilt es la biblioteca recomendada por Google para la inyección de dependencias en Android. Se configuró un módulo que provee las instancias de Retrofit, NewsProvider y NewsRepository de forma automática en toda la aplicación.

```

package com.example.myapplication.di

import ...

@Module
@InstallIn(...value = SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://newsapi.org/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }

    @Provides
    @Singleton
    fun provideNewsProvider(retrofit: Retrofit): NewsProvider {
        return retrofit.create(NewsProvider::class.java)
    }

    @Provides
    @Singleton
    fun provideNewsRepository(newsProvider: NewsProvider): NewsRepository {
        return NewsRepositoryImpl(newsProvider)
    }
}

```

7. ViewModel (Patrón MVVM)

El ViewModel actúa como intermediario entre la capa de datos y la UI. Expone el estado de forma reactiva mediante `StateFlow`, permitiendo que la interfaz se actualice automáticamente cuando los datos cambian.


```

2 Usages
@HiltViewModel
class NewsViewModel @Inject constructor(
    private val repository: NewsRepository
) : ViewModel() {

    2 Usages
    private val _articles = MutableStateFlow<List<Article>> ( value = emptyList() )
    1 Usage
    val articles: StateFlow<List<Article>> = _articles

    3 Usages
    private val _isLoading = MutableStateFlow ( value = false )
    1 Usage
    val isLoading: StateFlow<Boolean> = _isLoading

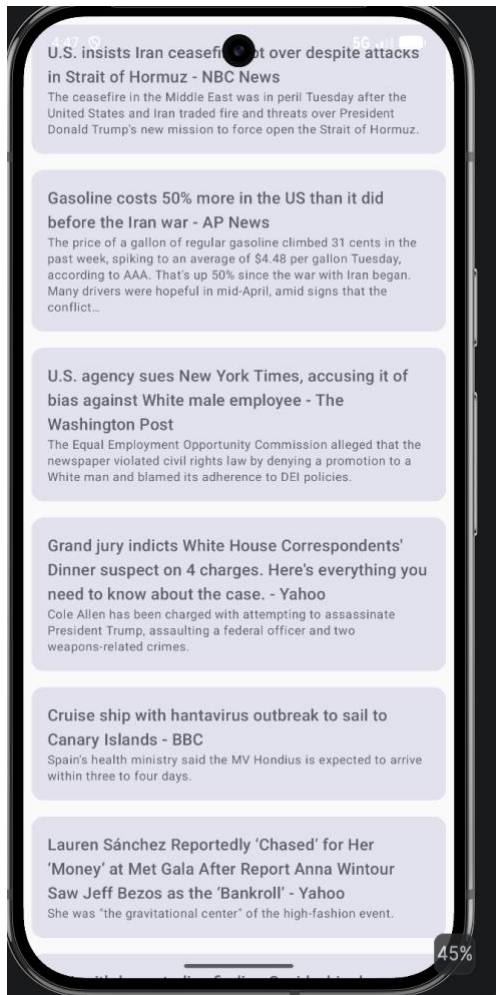
    init {
        loadNews()
    }

    1 Usage
    fun loadNews() {
        viewModelScope.launch {
            _isLoading.value = true
            try {
                _articles.value = repository.getTopHeadlines()
            } catch (e: Exception) {
                // manejar error
            } finally {

```

8. Interfaz de Usuario con Jetpack Compose

La UI se construyó completamente con Jetpack Compose, el toolkit moderno de Android para construir interfaces declarativas. Se usó LazyColumn para mostrar la lista de noticias y AsyncImage de Coil para cargar imágenes desde URL.



9. Pruebas Unitarias

Se implementaron pruebas unitarias usando JUnit4 y MockWebServer de OkHttp. MockWebServer permite simular respuestas de un servidor real sin necesidad de conexión a internet, garantizando pruebas reproducibles y confiables.

La prueba verifica que el repositorio parsea correctamente la respuesta JSON de la API y retorna la lista de artículos esperada.

```

class NewsRepositoryTest {

    5 Usages
    private lateinit var mockWebServer: MockWebServer
    2 Usages
    private lateinit var newsProvider: NewsProvider
    2 Usages
    private lateinit var repository: NewsRepository

    @Before
    fun setUp() {
        mockWebServer = MockWebServer()
        mockWebServer.start()

        val retrofit = Retrofit.Builder()
            .baseUrl(mockWebServer.url(path = "/"))
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        newsProvider = retrofit.create(NewsProvider::class.java)
        repository = NewsRepositoryImpl(newsProvider)
    }

    @Test
    fun `getTopHeadlines returns list of articles`() = runTest {
        // Simular respuesta del servidor
        val mockResponse = MockResponse()
            .setBody("""{"status":"ok","totalResults":1,"articles":[]}""")
            .setResponseCode(200)
        mockWebServer.enqueue(mockResponse)

        val result = repository.getTopHeadlines()
    }
}

```

✓ 1 test passed 1 test total, 317 ms

```

> Configure project :app
AGPBI: {"kind":"warning","text":"The option setting 'android.disableKotlinSourceSets=false' is experimental.\n\nThe current default"}
> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:generateDebugResources UP-TO-DATE
> Task :app:packageDebugResources UP-TO-DATE

```

✓ 1 test passed 1 test total, 488 ms

Executing tasks: [:app:testDebugUnitTest, --tests, com.example.myapplication.data.repository.NewsRepositoryTest] in project /User:

```

> Configure project :app
AGPBI: {"kind":"warning","text":"The option setting 'android.disableKotlinSourceSets=false' is experimental.\n\nThe current default"}
> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:generateDebugResources UP-TO-DATE

```

10. Conclusiones

El desarrollo de esta aplicación permitió aplicar los conceptos fundamentales de la arquitectura moderna de Android:

- El patrón MVVM separa claramente las responsabilidades entre la UI, la lógica de presentación y los datos.
- Hilt simplifica enormemente la inyección de dependencias, reduciendo el código boilerplate y mejorando la testabilidad.
- Retrofit facilita el consumo de APIs REST con una sintaxis clara y soporte nativo para corrutinas de Kotlin.
- Jetpack Compose permite construir UIs modernas y reactivas con menos código que el sistema de Vistas tradicional.
- Las pruebas unitarias con MockWebServer garantizan que la capa de datos funciona correctamente de forma aislada.

La combinación de estas tecnologías representa el estándar actual para el desarrollo profesional de aplicaciones Android, y su dominio es fundamental para cualquier desarrollador móvil.